

axiMini

경량 AI 기반 EDR

Lite AI-based Endpoint Detection & Response

guide.pdf · 8주 · 4–5 명 팀 과제

2일 축소판 · 1 인
동작하는 전체 파이프라인

정적 분석 · 실시간 감시 · 격리소 · 샌드박스 · 대시보드

무엇을 만들었나

What this does

감시

파일 생성 감시

Watched folder monitoring (watchdog)

AI

AI 정적 분석

Static ML scan (LightGBM, 2568-dim features)

동적

동적 분석

Dynamic analysis in a hardened Docker sandbox

격리

자동 격리

Automatic quarantine with recoverable vault

관제

실시간 관제

Live Korean SOC dashboard

주의: 실제 악성코드 없음 · No live malware in this project

전체 구조

End-to-end pipeline



정적 분석 엔진

Static scanner

파일 바이트 (최대 128 MiB)

↓

FastPEFeatureExtractor
→ 2,568-차원 float32 벡터 (EMBER2024 호환, 비트 단위 동일)

↓

LightGBM · 트리 2,063 개 · 모델 460ec1ca4d77…

↓

임계값 (threshold) — Val 데이터로 선택

T_low (샌드박스 진입)	**0.290884**	Val FPR 1.0%
T_high (자동 격리)	**0.999917**	Val FPR 0.1%

학습 데이터 (training data)

EMBER2024 Win32_test · 720,000 PE 파일 · 시간순 분할

train 1-8주 (480k) · val 9-10주 (120k) · test 11-12주 (120k)

실시간 감시 데몬

Real-time watcher daemon



- 파일 확장자가 아니라 내용(MZ)으로 PE 판정
- 핸들러에서 time.sleep() 금지 (이벤트 유실 방지)
- lost-update 레이스 방지: compare-and-swap on last_event

감지 → 판정 지연시간: 약 0.8 – 1.1 초

격리소 (Quarantine Vault)

Safe, recoverable, auditable

1

원본 파일 이름 변경 (같은 폴더)

rename → *.axiq-tmp (원자적)

2

SHA-256 재확인

검사 시점 바이트와 일치하는지 검증

3

XOR 중화 + 쓰기 전용 저장

<sha>.qbin (MAGIC | key | data^key)

4

원본 삭제 + 사이드카 JSON 기록

원본 경로, 점수, 모델, 판단 근거

복원 (Restore) — 오탐 (False Positive) 처리

① 허용 목록(allowlist)에 SHA-256 먼저 등록

② 그 다음 원래 경로에 복원 (재격리 레이스 방지)

데몬은 모든 판정 전에 allowlist를 먼저 확인

CLI

```
$ python -m axi.quarantine list
$ python -m axi.quarantine restore <sha-prefix>
$ python -m axi.quarantine delete  <sha-prefix>
```

동적 분석 샌드박스

Hardened Docker sandbox

Docker Container (axi-sandbox:1)

- `--network none`
  connect → ENETUNREACH
- `--read-only`
  write → EROFS
- `/work: noexec`
  execve → EACCES
- `--cap-drop ALL`
  cap syscall → EPERM
- `--pids-limit 32`
  fork → EAGAIN
- `--user 10001 (non-root)`
  privileged ops 거부

점수 계산 방식 (Scoring)

방어 장벽이 몇 개 건드려졌는가로 측정 (공격 패턴 매칭 X)

네트워크 차단 위반	`net_denied (ENETUNREACH)`	+25
실행 금지 위반	`exec_denied (EACCES)`	+25
쓰기 금지 위반	`rofs_denied (EROFS)`	+20
대량 파일 삭제	`mass_delete > 10`	+20
대량 파일 쓰기	`mass_write > 15`	+15
권한 요구	`eperm_denied (EPERM)`	+10

합산 점수: 30 미만 = 통과 · 30–69 = 의심 · 70+ = 악성

정상 샘플(benign_hello, benign_fileio)은 둘 다 0점

SOC 대시보드

Korean SOC Console (Streamlit)

axiMini SOC · 실시간 보안 관제

총 검사	통과	격리	샌드박스	평균 지연
10 건	6 건	2 건	1 건	1,273 ms

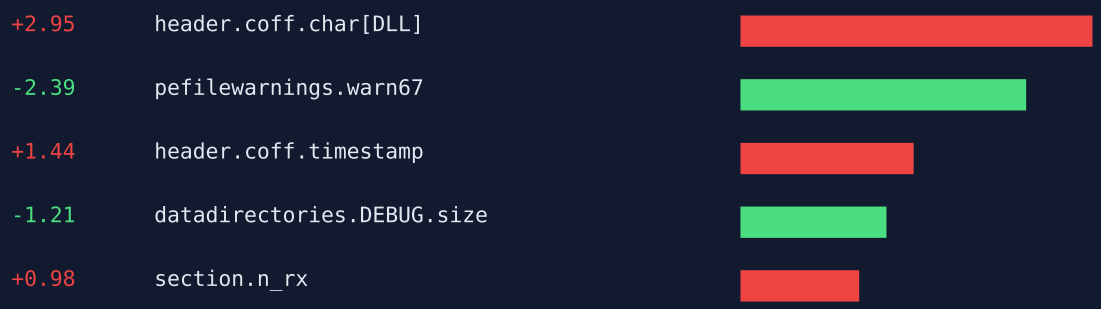
이벤트 상세 | 격리소 | 조치 기록 | 모델 & 지표

파일: watched₩getprocaddr32.exe

SHA-256: db0dc45b5711…

악성 점수: 0.977325 → 악성 (격리됨)

주요 판단 근거 (TreeSHAP)



□□□ (ms): read 1.9 · extract 63 · predict 1.3 · explain 438

□□: □□ □□ (quarantine/db0dc45b5711…qbin)

□□: 460ec1ca4d77… (□□ □□ □□)

SQLite 연결은 mode=ro — UI 버그가 증거를 조작할 수 없음

DB mtime 기반 캐싱으로 데몬 쓰기 직후 즉시 반영

결과 ① 모델 성능

Results · Model performance

Val (9-10주)

ROC-AUC 0.99885

pAUC(FPR≤0.5%) 0.970

FPR @ T_high 0.100%

Test (11-12주)

ROC-AUC 0.99816

pAUC(FPR≤0.5%) 0.958

FPR @ T_high 0.210%

[주의] 시간 드리프트 감지 (Concept Drift)

학습 2주 후: 오탐률이 2배로 상승 (0.10% → 0.21%)

이항 분포 기준 8.5σ — 노이즈가 아닌 실제 분포 변화

ROC-AUC는 거의 변하지 않지만(0.9989 → 0.9982)
운용 임계값에서의 FPR은 극적으로 변함

탐지 성능

92.6%

Test 세트 탐지율 (TPR)

0.21%

Test 세트 오탐률 (FPR)

목표: FPR ≤ 0.1% · 달성: val V test X (드리프트)

[핵심] 가장 중요한 결과는 "드리프트가 실제로 측정됐다"는 것

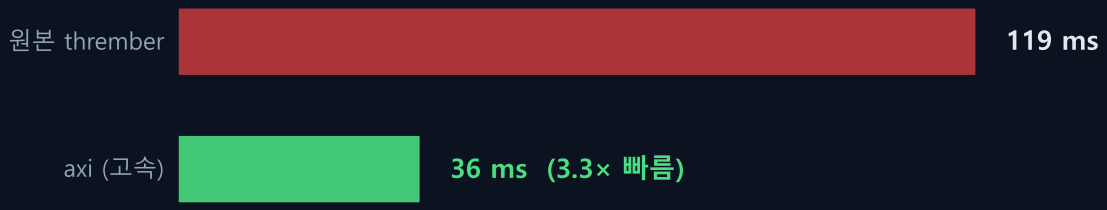
생산 환경에서는 안전 여유, 주기적 재학습, 서명 허용 목록으로 완화

(개념만 설명 — 재학습 파이프라인은 범위 밖, 설계는 로그 11)

결과 ② 속도

Results · Latency

파일 1개 스캔 시간 (ms, p50 중앙값)



주요 최적화 (출력은 비트 단위 동일)

① 문자열 regex: S×R → R (finditer + searchsorted)

② 엔트로피: Counter → bincount (첫 등장 순서 보존)

③ gc.freeze(): pefile close()의 전체 GC 비용 제거

실시간 탐지 지연 (파일 저장 완료 → DB 기록)

약 0.8 – 1.1 초

디바운스 0.5s + 안정성 0.2s + 추출 + 예측

[검증] 비트 단위 동일성 검증

학습 시 쓴 추출기 vs 스캔 시 쓴 고속 추출기 = 동일 바이트

실제 PE 1,008 개 + 합성 엣지 케이스 8 개 → 불일치 0

음성 통제: 엔트로피를 1 ulp 바꾸면 모두 실패 (검증기 무의미하지 않음)

guide.pdf 대비 주요 차이점

Deliberate deviations from the spec

무작위 분할 → 시간순 분할

guide: random split (미래 유출) · axi: by week (실제 배포 조건)

Test 세트 임계값 → Val 세트 임계값

guide: threshold=0.97 on test · axi: val FPR로 선택, test는 1회 측정만

정확도 → Partial AUC

50/50 데이터에서 accuracy/precision 오해 유발 → FPR ≤ 0.5% 구간만 평가

확장자 라우팅 → 내용 라우팅

update.bin 으로 이름 바꾼 PE 도 정상 검사

핸들러 sleep() → 즉시 반환

OS 이벤트 버퍼 오버플로우 방지 (Debouncing을 별도 스레드로)

샌드박스 서명 매칭 → 방어 장벽 측정

공격 패턴 DB 유지 비용 없음, 새로운 기법에도 강건

대시보드에 삭제 버튼 → 읽기 전용

UI 버그가 증거를 조작할 수 없게 함 (CLI가 유일한 writer)

공통 원칙: "측정 가능한 방어적 설계" 우선

요약

Summary

2일간 완성한 동작하는 Lite AI-EDR

8주 팀 프로젝트의 핵심 파이프라인을 1인 범위로 구현

정적 분석 · 실시간 데몬 · 격리 · 샌드박스 · 대시보드

34개 테스트 통과 · 720,000 PE 파일 학습

19	34	720k	2,063	2,568
학습 로그	자동 테스트	학습 데이터	LightGBM 트리	특징 차원

남은 과제 (Open items)

- 스캔 p95 < 50 ms 미달 (pefile 전체 파싱 유지 선택)
- 데몬 ↔ 샌드박스 연동 (ELF 라우팅)
- YARA 룰 자동 생성기 (docs/YARA_RULE_GENERATOR.md에 설계 완료)
- 서명자 기반 허용 목록, 재학습 파이프라인

학습한 것

· ML 평가 (시간순 분할, 임계값 선택, 드리프트 측정)

· 실시간 시스템 설계 (디바운스, 레이스 방지, 백프레셔)

· 컨테이너 격리 (Linux 네임스페이스, cap-drop, strace)

· 증거 처리 (원자적 이동, 감사 로그, 읽기 전용 UI)

감사합니다 · Thank you